

SGLang-Omni 每日 PR 学习日报

2026-06-21 · 覆盖过去 24 小时内合并 (merged) 的 PR

4

合并的 PR

4

主题分类

3

含代码改动

1

架构设计文档

Omni / 多模态 / 稳定性 (CUDA Graph)

Omni 多阶段架构 / 设计文档 (RFC)

TTS / 多阶段流水线 / 吞吐调优

多模态 / ASR 预处理优化

仓库: [sgl-project/sglang-omni](https://github.com/sgl-project/sglang-omni) · 讲解深度优先: 每个 PR 都按「概览 / 解决什么问题 / 改了什么 / 知识点 / 延伸思考」五段展开, 概念采用「是什么→为什么→本 PR 怎么用」三段式。

本期目录

- **#833** — Extract Qwen3-ASR mel at true audio length instead of padding to 3000 mel frames (多模态 / ASR 预处理优化 · @0xjeffro)
- **#830** — [Ming]: Use thread_local capture_error_mode for CUDA graph (Omni / 多模态 / 稳定性 (CUDA Graph) · @AkazaAkane)
- **#756** — [Higgs TTS] Raise AR server default to 64 and expose standard batch knobs (TTS / 多阶段流水线 / 吞吐调优 · @estellaliu233)
- **#537** — [Docs] Internalize RFC comments + consolidate historical RFCs (#488 part 4.1 + 4.2) (Omni 多阶段架构 / 设计文档 (RFC) · @JiaxinD)

PR #833



Extract Qwen3-ASR mel at true audio length instead of padding to 3000 mel frames

1. PR 概览

编号	#833
标题	Extract Qwen3-ASR mel at true audio length instead of padding to 3000 mel frames
作者	@0xjeffro
状态	merged
主题	多模态 / ASR 预处理优化
链接	https://github.com/sgl-project/sglang-omni/pull/833

一句话概括：让 Qwen3-ASR 的音频特征提取按音频的真实时长计算 mel 频谱，而不是无脑补齐到 30 秒，端到端吞吐 **+24%**。

2. 它解决了什么问题

Qwen3-ASR（语音识别模型）复用了 OpenAI **Whisper** 的特征提取器（feature extractor）。Whisper 的默认行为是：不管你给它一段多长的音频，它都会先把音频补零（padding）到 30 秒，再去算 mel 频谱（mel spectrogram）。

问题来了：基准测试用的 SeedTTS 语料每条只有 4~7 秒。也就是说，每来一条请求，CPU 都要花力气对一大段“静音”做 FFT（快速傅里叶变换）算频谱——纯属浪费。作者通过 profiling 发现，单条请求的预处理（`request_build`）要花约 **28ms**，而且它是串行跑在 GPU 之前的，等于卡住了整条流水线的喉咙。

关键洞察：Qwen3-ASR 的音频编码器（encoder）本来就是变长（**variable-length**）的，它会用 `feature_attention_mask` 把补出来的无效帧丢掉。既然下游本来就会丢弃 padding，那上游这 30 秒的 padding 就是“算了又扔”的纯浪费。

3. 具体做了什么改动

核心文件只有一个：`sglang_omni/models/qwen3_asr/request_builders.py`。

改动其实只有两行参数：

```
extracted = feature_extractor(
    audio,
    sampling_rate=_SAMPLE_RATE,
    return_tensors="pt",
    return_attention_mask=True,
    padding="longest",    # 新增：按真实长度而不是补到 30s
    truncation=True,      # 新增：超长才截断
)
features = extracted.input_features # [128, true_frames] (<= 3000)
feature_attention_mask = getattr(extracted, "attention_mask", None)
```

逐行解释这次改动的灵魂——那两个新参数：

- `padding="longest"`：Whisper 默认是 `padding="max_length"`，意思是“补到最大长度（3000 帧 ≈ 30s）”。改成 `"longest"` 后，只补到“这一批里最长的那条”的长度。当一批里只有一条短音频时，几乎不补，mel 帧数 = 真实帧数。
- `truncation=True`：万一遇到超过 30s 的音频才截断，保证不越界。

注释里有一句很重要的安全说明：为什么这么做对 Qwen3-ASR 安全，但对原版 Whisper 不安全？因为原版 Whisper 的编码器是定长的（它硬性要求 3000 帧输入），你给它变长输入会直接崩；而 Qwen3-ASR 的编码器是变长的，靠 mask 选有效帧，所以可以安全地喂变长 mel。结果对比注释也很直白：

```
-         features = extracted.input_features # 128, 3000
+         features = extracted.input_features # [128, true_frames] (<=
```

效果（单卡 H200，SeedTTS EN 1088 条，并发 32，3 次重复取平均）：吞吐 22.43 → 27.88 samples/s (+24.3%)，`request_build` 阶段 27.9ms → 18.5ms (-34%)，而 WER（词错率）基本不变（在噪声范围内）——免费的提速。

4. 涉及的知识点

Mel 频谱 / FFT (梅尔频谱与快速傅里叶变换)

它是什么：声音是一维波形（每秒上万个采样点）。模型不直接吃波形，而是先把它切成小窗口，用 FFT 把每个窗口转成“频率分布”，再按人耳听觉特性压缩到 mel 刻度，得到一张二维“频谱图”（这里是 128 个 mel 通道 × 若干时间帧）。为什么需要：频谱图更紧凑、更接近人耳感知，是几乎所有语音模型的标准输入。

本 PR 里：算 FFT/mel 是 CPU 密集操作，对 30s 静音算 mel 就是把算力烧在空气上；只算真实长度直接省掉这部分。

Whisper Feature Extractor 的 padding 策略

它是什么：HuggingFace 里把原始音频转成 mel 的工具。它有 `padding` 参数：`max_length`（补到固定 3000 帧）、`longest`（补到本批最长）。类比：寄快递时，`max_length` 是“不管东西多小都塞进一个标准大箱子再发”，`longest` 是“按这批里最大的那件配箱”。

本 PR 里：从大箱子换成了刚好够用的箱子。

attention mask (注意力掩码) 与变长编码器

它是什么：一个 0/1 数组，告诉模型“哪些位置是真数据、哪些是补出来的，请忽略后者”。

为什么需要：批处理 (batching) 要求同一批张量形状一致，于是必须补齐；mask 让模型在计算注意力时跳过补出来的部分。

本 PR 里：因为 Qwen3-ASR 用 `feature_attention_mask` 选有效帧，所以“少补一点”不会影响正确性——这是这次优化成立的前提。

Profiling 与“串行瓶颈”

它是什么：profiling 是给程序“做体检”，量出每个阶段耗时。

本 PR 里：作者发现 `request_build`（建请求/做预处理）跑在 GPU 之前且是串行的，所以即便 GPU 很快，这一步也会拖慢整条流水线。优化串行前置步骤，收益会直接体现在端到端延迟上。

RTF / WER (实时率与词错率)

RTF (Real-Time Factor) : 处理时长 ÷ 音频时长。RTF=0.25 表示处理 1 秒音频只花 0.25 秒，越小越好。

WER (Word Error Rate) : 识别文本与标准答案的词级错误比例，越低越好。
本 PR 里：用来证明“提速没有牺牲质量”——RTF 降了 19%，WER 基本不动。

5. 我能学到什么 / 延伸思考

- 可复用工程思想：① “沿用默认值”常常是隐藏的性能税——别人为通用场景设的默认（补到 30s）未必适合你的场景（短音频）。② 优化要找串行的、前置的热点，那里的每一毫秒都直接进端到端延迟。③ 改动越小、越聚焦越好（这里只改了两个参数），但必须想清楚“为什么对我安全”（变长编码器 + mask）。
- 延伸学习：Whisper 的输入处理、mel 频谱原理、HuggingFace `FeatureExtractor` 的 padding/truncation 语义、以及“变长 vs 定长编码器”的区别。

PR #830



[Ming]: Use thread_local capture_error_mode for CUDA graph

1. PR 概览

编号	#830
标题	[Ming]: Use thread_local capture_error_mode for CUDA graph
作者	@AkazaAkane
状态	merged
主题	Omni / 多模态 / 稳定性 (CUDA Graph)
链接	https://github.com/sgl-project/sglang-omni/pull/830

一句话概括：给 Ming-Omni talker 的两处 CUDA Graph 捕获加上 `capture_error_mode="thread_local"`，让它在多线程同进程（colocated）服务下捕获时不会被“邻居线程”的无关 CUDA 操作搞崩。

2. 它解决了什么问题

先理解场景：SGLang-Omni 把多个子系统（比如 talker、vocoder 等）放在同一个进程里、同一张 GPU 上一起跑（叫 **colocated serving** / 共置服务）。这些子系统在不同线程里各干各的，但共享同一个 CUDA 上下文。

CUDA Graph 有一个“捕获（**capture**）”阶段——它要把一串 GPU 操作录制成一张可重放的“图”。捕获期间，PyTorch 默认用全局（**global**）的捕获错误模式：只要它检测到“这条 CUDA 流上有别的、和本次捕获无关的 CUDA 活动”，就可能直接判定捕获失败并报错。

问题：Ming 的两处图捕获（`CFMGraphExecutor.initialize_graph()` 和 `MingOmniTalker.generate()`）不只在“纯启动预热”时发生，还可能在运行中懒加载（**lazy**）触发。如果此时同进程的兄弟线程正在做别的 CUDA 工作，全局错误模式就会让捕获“被无辜牵连”而失败。这和 #825（MOSS vocoder）、#829 是同一类 bug。

3. 具体做了什么改动

改动极小但精准——给两处 `torch.cuda.graph(...)` 显式传入

`capture_error_mode="thread_local"` :

```
# CFMGraphExecutor._initialize_graph()
self.graph = torch.cuda.CUDAGraph()
try:
    with torch.cuda.graph(self.graph, capture_error_mode="thread_local"):
        self.gen_lat_placeholder = self.cfm.sample(...)

# MingOmniTalker.generate()
with torch.cuda.graph(model_graph, capture_error_mode="thread_local"):
    outputs_placeholder = self.model(
        position_ids=position_ids_placeholder,
        cache_position=cache_position_placeholder,
        ...
    )
```

逐点解释：`capture_error_mode` 控制“捕获期间如何看待其它 CUDA 活动”。默认 `"global"` = “整条流/全进程只要有别的活动就算错”；改成 `"thread_local"` = “只在意本线程的违规操作，别的线程在干嘛我不管”。对共置服务来说，这正是想要的语义：本地图捕获不该因为别的子系统在忙而失败。

这个改动不会改变任何计算结果——它只动了“捕获时的错误检查范围”，张量数值、采样、解码逻辑都不变，因此无需精度测试。

更有意思的是它配的单元测试：不真的去跑 GPU（CI 不一定有卡），而是用 Python 的 `ast` 模块解析源码，断言这两个方法里所有 `torch.cuda.graph(...)` 调用都带上了 `capture_error_mode="thread_local"` :

```
def _has_thread_local_capture_error_mode(call: ast.Call) -> bool:
    return any(
        keyword.arg == "capture_error_mode"
        and isinstance(keyword.value, ast.Constant)
        and keyword.value.value == "thread_local"
        for keyword in call.keywords
    )

def test_ming_lazy_cuda_graph_captures_use_thread_local_error_mode() -> bool:
    tree = ast.parse(_TALKER_SOURCE.read_text())
    for class_name, method_name in [
        ("CFMGraphExecutor", "_initialize_graph"),
        ("MingOmniTalker", "generate"),
    ]:
        method = _method_node(tree, class_name, method_name)
        graph_calls = _torch_cuda_graph_calls(method)
        assert graph_calls
        assert all(_has_thread_local_capture_error_mode(c) for c in graph_calls)
```

4. 涉及的知识点

CUDA Graph (CUDA 图)

它是什么：把一连串 GPU kernel 调用“录制”成一张图，之后一次性重放 (replay)，省掉每次单独从 CPU 提交 kernel 的开销 (launch overhead)。

为什么需要：自回归解码每步只算一点点，CPU 提交开销占比很大；用图把整步“打包重放”能显著提速，是 LLM 推理的标配优化。

本 PR 里：Ming talker 用图来加速；问题出在“录制”这一时刻的健壮性，而非重放。

capture (捕获) vs replay (重放)

它是什么：图有两个阶段——捕获（录制一次，期间对环境很敏感）和重放（之后反复执行，很快）。

类比：捕获像“录节目”，录的时候片场不能乱；重放像“播录像”，随便放。

本 PR 里：只加固了“录节目”阶段的容错，让录制不被隔壁剧组干扰。

colocated serving (共置服务) / 多线程同进程 + 同一 CUDA 上下文

它是什么：把多个模型/阶段塞进同一个进程、同一张 GPU，用多线程并行，以省显存、省机器。

为什么需要：Omni 流水线有很多小阶段，单独各占一进程/一张卡太浪费。

本 PR 里：正因为共享 CUDA 上下文，默认的 global 错误模式才会“误伤”——thread_local 把判定范围缩小到本线程，是共置场景的正确选择。

lazy / 运行时捕获 vs 启动预热捕获

它是什么：理想情况下图都在启动预热（warmup）时一次性捕获好，环境干净；但有些路径会在第一次真正用到时才懒捕获。

本 PR 里：懒捕获可能发生在服务已经在跑、别的线程正忙的时候，这正是需要 `thread_local` 的高风险时刻。

用 AST 写“源码级”单元测试

它是什么：`ast`（抽象语法树）把 Python 代码解析成结构化的树，可以在不运行代码的情况下检查“代码长什么样”。

为什么需要：这条规则（所有图捕获必须带 `thread_local`）很难用普通运行测试覆盖（要真 GPU、要构造并发冲突）；用 AST 直接断言源码符合规范，又快又稳，还能防止未来有人新增捕获点时忘了加参数。

本 PR 里：这是一种把“团队约定”固化成自动化护栏的轻量手法。

5. 我能学到什么 / 延伸思考

- 可复用工程思想：① 共享资源（CUDA 上下文）下，默认的“全局判定”往往太严，要把作用域收窄到“本线程/本任务”。② 一个“一行参数”的修复，背后是对运行时时序（什么时候、在哪个线程发生）的深刻理解。③ 当行为难以用运行时测试覆盖时，用静态分析（AST）给约定上锁是性价比极高的护栏。
- 延伸学习：PyTorch CUDA Graph API（`torch.cuda.graph` / `CUDAGraph` / `capture_error_mode`）、CUDA stream 与上下文、Python `ast` / `inspect` 模块。可顺带看相关的 #825、#829。

PR #756



[Higgs TTS] Raise AR server default to 64 and expose standard batch knobs

1. PR 概览

编号	#756
标题	[Higgs TTS] Raise AR server default to 64 and expose standard batch knobs
作者	@estellaliu233
状态	merged
主题	TTS / 多阶段流水线 / 吞吐调优
链接	https://github.com/sgl-project/sglang-omni/pull/756

一句话概括：把 Higgs TTS 的自回归（AR）服务默认并发从 16/16 提到 **64/64**，并把 `--max-running-requests` / `--cuda-graph-max-bs` 做成所有模型通用的标准 CLI 调参开关；高并发吞吐提升约 **+38%**（c32） / **+14%**（c64）。

2. 它解决了什么问题

Higgs TTS 之前的 AR 服务默认值很保守：`max_running_requests=16`、`cuda_graph_max_bs=16`（记作 16/16）。在高并发压测下，这个上限把吞吐卡死了——客户端来了一堆请求，但服务器一次最多只让 16 个并行解码，多出来的只能排队。

更糟的是，调这两个参数的方式当时是 **Higgs** 专用的小工具，别的模型（Qwen3-Omni、MOSS、Voxtral……）不能复用同一套开关，调参体验割裂。

还有一个隐藏 bug：Higgs 模型内部的采样器/CUDA Graph 缓冲区是按“旧的写死的最大批大小”分配的。当你想用更大的捕获尺寸（比如 128）时，模型内部 buffer 形状对不上，捕获会直接报错（batch shape mismatch）。

3. 具体做了什么改动

这是一个“接口统一 + 默认值上调 + 顺手修 bug”的组合拳，涉及多个文件。

(a) 让模型内部 **buffer** 跟着服务配置走（修 128 捕获 bug），文件 `models/higgs_tts/model.py`：不再用写死的 `_DEFAULT_MAX_BATCH_SIZE=64`，而是直接去问 SGLang 全局配置：

```
def _resolve_max_running_requests() -> int:
    try:
        from sglang.srt.server_args import get_global_server_args
        # 直接问 SGLang：本次服务的 max_running_requests 是多少？
        return int(get_global_server_args().max_running_requests)
    except (ImportError, AttributeError, TypeError, ValueError) as exc:
        fallback = 64
        logger.warning(
            f"Falling back to Higgs max_running_requests={fallback} because {exc}"
            f"SGLang global server args are unavailable: {exc}"
        )
        return fallback

# __init__ 里不再用写死的 max_batch_size 参数：
self._max_batch_size = _resolve_max_running_requests()
pool_size = self._max_batch_size + 1
self._sampler_pool = HiggsBatchedSamplerState(max_batch_size=pool_size,
```

这样 Higgs 内部采样器池（sampler pool）和 CUDA Graph buffer 的大小，就和 SGLang 实际的 `max_running_requests` 对齐了——你设多大它就开多大，128 也能正确捕获。

(b) 新增两个通用 CLI 开关，文件 `cli/serve.py`：

```
# sglang_omni/cli/serve.py — 新增两个通用 CLI 开关
max_running_requests: Annotated[int | None, typer.Option(
    "--max-running-requests", min=1,
    help="Override SGLang generation stage max_running_requests.")]] = None
cuda_graph_max_bs: Annotated[int | None, typer.Option(
    "--cuda-graph-max-bs", min=1,
    help="Override SGLang generation stage cuda_graph_max_bs.")]] = None,

# 把开关写进"generation 角色"对应的那个 stage
generation_server_args_overrides: dict[str, object] = {}
if max_running_requests is not None:
    generation_server_args_overrides["max_running_requests"] = max_running_requests
if cuda_graph_max_bs is not None:
    generation_server_args_overrides["cuda_graph_max_bs"] = cuda_graph_max_bs
if generation_server_args_overrides:
    generation_stage_name = (
        type(merged_config).generation_sglang_role_to_stage().get("generation")
    )
    if generation_stage_name is None:
        _raise_unsupported_flag(merged_config, "--max-running-requests/--cuda-graph-max-bs",
        _apply_stage_server_args_override(
            merged_config, stage_name=generation_stage_name,
            updates=generation_server_args_overrides,
            reason="SGLang generation server args override")
    )
```

这里的关键设计是“角色 → stage”的间接映射：CLI 不直接写死“改 tts_engine”，而是问当前模型的配置“你的 `generation` 角色对应哪个 stage？”，再把覆盖值写进那个 stage。这样同一套开关对所有模型都好使。

(c) 每个模型声明自己的 `generation` 角色，文件 `config/schema.py` 加基类方法 + 各模型 `config.py` 实现：

```
# 每个模型的 config.py 声明：哪个 stage 扮演 "generation" 角色
# Higgs / MOSS / Qwen3-TTS / S2-Pro -> "tts_engine"
class HiggsTtsPipelineConfig(PipelineConfig):
    @classmethod
    def generation_sglang_role_to_stage(cls) -> dict[str, str]:
        return {"generation": "tts_engine"}

# Qwen3-Omni 的 AR talker -> "talker_ar"
class Qwen3OmniSpeechPipelineConfig(PipelineConfig):
    @classmethod
    def generation_sglang_role_to_stage(cls) -> dict[str, str]:
        return {"generation": "talker_ar"}
```

(d) 上调默认值并清理重复常量：删掉散落各处的

`DEFAULT_MAX_CONCURRENCY=16`，让 stage 构造函数的默认值（64/64）成为唯一事实来源（single source of truth）。如果只设了两个开关里的一个，另一个自动取相同值，降低误用。

实测 (H200, SeedTTS EN 1088) : **32/32** 相比 **16/16** 在并发 32 下吞吐 **+38.3%**、中位延迟 **-28.5%** ; **64/64** 把高吞吐区间延伸到并发 64 (c64 吞吐 +14%) , 启动时间几乎不变, 显存只多约 236 MiB。WER 无回退。最终选 **64/64** 作默认——比 128/128 更省、收益却接近。

4. 涉及的知识点

Autoregressive (AR, 自回归) 生成 / talker

它是什么: 像 GPT 一样“一个 token 一个 token 往外吐”, 每一步都依赖上一步的输出。TTS 里负责生成语音 token 的模块常叫 talker。

为什么需要: 语音/文本本质是序列, 自回归能建模强时序依赖。

本 PR 里: 被调优的就是这个 AR 生成阶段 (Higgs 的 tts_engine、Qwen3-Omni 的 talker_ar) 。

max_running_requests (最大并发请求数)

它是什么: SGLang 调度器同一时刻最多让多少个请求一起解码。

为什么需要: 它是 continuous batching (连续批处理) 的“批大小上限”——开大能提升吞吐, 但更吃显存。

本 PR 里: 从 16 提到 64, 让服务器能同时招纳更多并发请求, 吞吐随之上去。

continuous batching (连续批处理)

它是什么: 传统批处理要凑齐一批一起算、一起结束; 连续批处理则是“谁算完谁先走, 空出的位置立刻补进新请求”, 让 GPU 时刻满载。

类比: 不是“一桌人到齐才上菜、吃完一起走”, 而是“有空位就立刻安排下一位”的流水席。

本 PR 里: max_running_requests 决定这张“流水席”最多能坐多少人。

cuda_graph_max_bs (CUDA Graph 最大捕获批大小)

它是什么: 为加速解码, SGLang 会为不同 batch size 预先捕获 CUDA Graph。这个参数是“捕获到多大的批”。

为什么需要: 只有 batch size $\leq$ 这个值时才能走快的图重放路径; 否则退回较慢的 eager 执行。

本 PR 里: 通常和 max_running_requests 设成一样大 (所以默认 64/64) , 保证高并发时仍走图加速; 只设一个时自动补另一个。

CUDA Graph buffer / sampler pool (采样器池) 形状匹配

它是什么：捕获图时要用固定形状的占位张量 (placeholder buffers)。模型内部为采样准备的 buffer 必须 $\geq$ 实际批大小，否则捕获时形状对不上就崩。

本 PR 里：把内部 buffer 大小从“写死”改成“读 SGLang 配置”，从而修好了 128 捕获的 shape mismatch。

多阶段流水线里的“角色映射”与 single source of truth

它是什么：Omni 模型由多个 stage 串成流水线

(preprocessing $\rightarrow$ audio_encoder $\rightarrow$ tts_engine $\rightarrow$ vocoder...)。不同模型的 stage 名字不一样，但都有一个“负责生成”的阶段。用

`generation_sglang_role_to_stage()` 把抽象角色映射到具体 stage 名。

为什么需要：让“调参开关”这种通用功能不必为每个模型写一遍；删掉重复的默认常量，让默认值只在一处定义 (single source of truth)，避免“改了一处忘了另一处”。

本 PR 里：这是从“Higgs 专用补丁”升级为“全模型通用能力”的关键抽象。

5. 我能学到什么 / 延伸思考

- 可复用工程思想：① 默认值要有数据支撑——作者做了 16/32/64/128 的扫描 (sweep)，用吞吐/延迟/WER/显存/启动时间多维证据选出 64，而不是拍脑袋。
② 把“一次性补丁”重构成“通用机制” (角色映射 + 统一 CLI)，让所有模型受益。
③ 消灭重复常量、建立单一事实来源，是降低长期维护成本的经典做法。
④ 调吞吐时要同时盯吞吐/延迟/显存/精度四个维度，别只看一个数。
- 延伸学习：SGLang 的调度器与 ServerArgs、continuous batching、CUDA Graph 捕获机制、显存预算 (mem_fraction_static)、以及“并发 vs 吞吐 vs 延迟”的权衡曲线。

PR #537



[Docs] Internalize RFC comments + consolidate historical RFCs (#488 part 4.1 + 4.2)

1. PR 概览

编号	#537
标题	[Docs] Internalize RFC comments + consolidate historical RFCs (#488 part 4.1 + 4.2)
作者	@JiaxinD
状态	merged
主题	Omni 多阶段架构 / 设计文档 (RFC)
链接	https://github.com/sgl-project/sglang-omni/pull/537

一句话概括：把散落在 Lark 导出的 RFC 评审意见沉淀进仓库内的设计文档，并把 36 个塑造架构的历史 PR 整理成一条时间线式的“设计决策史”——这是理解整个 Omni 多阶段架构演进的最佳地图。

2. 它解决了什么问题

这是一个纯文档/知识管理类 PR，但对“想看懂这个仓库架构”的人价值极高。痛点是：项目早期的架构讨论和评审意见散落在外部工具（Lark/飞书）导出的文档里，仓库里的 `docs/design/refactor_rfc.md` 还留着一堆“待办（Pending）”占位，和 `main` 分支早已落地的实现对不上。新人很难从代码反推“当初为什么这么设计”。

这个 PR 做的就是“把外部讨论内化进仓库 + 把历史决策串成时间线”，让设计文档与现状一致，并成为一份可长期保存的架构编年史。

3. 具体做了什么改动（文档层面）

没有功能代码，主要是文档重写与整合：

- 把 **Pending** 块改写成已落地的结论：§1.2/§1.3/§1.4 里关于 hook 拆分、payload 改名、`stage_workers.py` 合并的“待办”，都已在 #558 实现，于是改写成定稿描述。

- 补回系统总览图（**mermaid** 图）：恢复 HTTP-API ↔ Client 的连线和 WebSocket 入口；澄清“没有应用层 TLS”（uvicorn 跑明文 HTTP，TLS 由反向代理/负载均衡终结）。
- 建立“设计决策史”：把 36 个架构相关 PR 按创建时间整理成两条主线——
 - ① 框架 / **V1** 重构线：原始 RFC 系列 + #496（拓扑感知的 LOCAL_OBJECT / CUDA-IPC 传输）+ #558（hook/payload/stage-worker 重构）+ #589（stage fusion，阶段融合）+ #824（统一采样种子）。
 - ② 模型接入 + **TTS** 优化线：#437（Ming-Omni V1）、#446（LLaDA2.0-Uni）、#451（Qwen3-TTS / Voxtral）、Higgs Audio v3 系列、MOSS-TTS 系列。
- 资产瘦身：用自包含的 SVG 示意图替换 1.4MB 的 PNG。

4. 涉及的知识点（重点：**Omni** 多阶段架构全景）

多阶段流水线（**multi-stage pipeline**）/ **stage + scheduler**

它是什么：Omni 模型（同时处理文本/音频/图像）不是一个大模型一口吃下，而是拆成多个 **stage**（阶段）串成流水线，例如 ASR：预处理 → 音频编码器 → 生成；TTS：预处理 → 音频编码 → AR 引擎 → vocoder。每个 stage 有自己的 scheduler（调度器）和 batching 策略。

为什么需要：不同阶段的特性差异巨大（CPU 密集的预处理 vs GPU 密集的解码 vs 变长的编码器），拆开后能各自独立批处理、独立扩缩、独立放卡。

本 PR 里：这正是这份 RFC 文档要讲清楚的主干架构。

SGLang 核心组件（scheduler / tokenizer manager / detokenizer / model runner / attention backend）

scheduler（调度器）：决定每一步把哪些请求组成一批送去算，是 continuous batching 的大脑。

tokenizer manager：在请求入口把文本/多模态输入转成 token，并管理请求生命周期。

detokenizer：把模型吐出的 token 转回文字/结果，常单独成进程以免阻塞主循环。

model runner：真正在 GPU 上跑前向（forward）的执行器，管理权重、KV cache、CUDA Graph。

attention backend：注意力的具体实现（如 FlashAttention、RadixAttention 等），决定速度与显存。

本 PR 里：Omni 把这些组件按 stage 组合复用，文档把它们的协作关系画进系统图。

KV cache / RadixAttention (前缀复用)

KV cache : 自回归生成时, 把已经算过的 token 的 Key/Value 缓存下来, 避免每步重算整段历史——这是 LLM 推理省时的核心。

RadixAttention : SGLang 的招牌特性, 用一棵基数树 (radix tree) 把相同前缀的 KV cache 共享/复用 (比如很多请求用同一段系统提示或同一段参考音频前缀)。

本 PR 里: 决策史中 Higgs 用 `Req.extra_key` 把 radix cache 按参考音频做命名空间隔离, 就是 RadixAttention 在 TTS 场景的应用。

LOCAL_OBJECT / CUDA-IPC 数据传输 (#496)

它是什么: 流水线相邻 stage 之间要传数据 (比如编码器输出的张量传给生成阶段)。如果跨进序列再反序列化会很慢; **CUDA-IPC** 让同机不同进程直接共享显存指针, 张量不出 GPU 就能交接; **LOCAL_OBJECT** 则是“拓扑感知”地选择最省的本地传输方式。

类比: 不是把货物装箱寄快递 (序列化), 而是直接把仓库钥匙递过去 (共享指针)。

本 PR 里: 这是多阶段架构性能的关键基础设施, 被记入决策史。

stage fusion (阶段融合, #589)

它是什么: 把原本分开的若干小 stage 合并成一个, 减少阶段间的数据搬运和调度开销。

为什么需要: 阶段拆得越细越灵活, 但每道边界都有传输/排队成本; 当两个阶段总是连在一起且都很轻时, 融合更划算。

本 PR 里: 体现了“先拆解获得灵活性, 再按数据择机融合”的演进节奏。

hook / payload / StagePayload 重构 (#558)

它是什么: **StagePayload** 是在流水线里各 stage 之间流转的“数据包”; hook 是在 stage 处理前后插入逻辑的钩子。#558 把 hook 拆分、payload 改名、合并 stage_worker。

本 PR 里: 你在 #833/#756 里看到的 `StagePayload`、`create_*_executor` 工厂、`factory_args`, 都是这套数据平面 (data-plane) 设计的产物——读懂这份决策史, 就能看懂前几个 PR 的代码长那样的原因。

RFC 与“设计决策史”这种工程文化

它是什么：RFC (Request for Comments) 是“先写设计、公开评审、再实现”的协作方式。把评审意见和历史决策沉淀进仓库，形成可追溯的编年史。

为什么需要：代码只告诉你“现在是什么”，决策史告诉你“为什么变成这样、当初放弃了哪些选项 (won't-do) ”，对新人 and 长期维护至关重要。

本 PR 里：把外部工具里的讨论内化、与代码现状对齐，是高质量开源项目的标志性实践。

5. 我能学到什么 / 延伸思考

- 可复用工程思想：① 文档要和代码同步演进——“待办占位”一旦落地就应改写成结论，否则文档就是债。② 把决策史串成时间线，是给团队和未来的自己留的“地图”。③ 知识不应锁在外部工具里，应内化进仓库、可被 grep、可随代码评审。
- 延伸学习（强烈建议作为读这个仓库的起点）：直接去读 `docs/design/refactor_rfc.md`；按决策史顺着 #496（传输）→ #558（数据平面）→ #589（融合）补齐前置知识；再回头看 SGLang 本体的 scheduler / RadixAttention / model runner 设计。先建立架构全景，再看单个 PR 的代码会轻松很多。